

Java 项目面试 Q&A

基于简历项目，整理 Java 后端高频面试题，重点覆盖 Spring Cloud 微服务、高并发、缓存、消息队列等

一、Spring Boot 与 Spring Cloud

Q1: Spring Boot 自动装配原理是什么？

参考回答：

核心注解 `@SpringBootApplication` 包含 `@EnableAutoConfiguration`，其流程：

1. 通过 `SpringFactoriesLoader` 读取 `META-INF/spring.factories`（Spring Boot 3.x 改为 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`）中配置的自动装配类
2. 根据 `@Conditional` 系列注解（`@ConditionalOnClass`、`@ConditionalOnMissingBean` 等）条件过滤
3. 满足条件的装配类被加载到 IoC 容器

项目关联：项目中用 Nacos 配置中心，其自动装配就是通过 `spring.factories` 注册 `NacosConfigAutoConfiguration` 实现的。

Q2: Spring Cloud 组件有哪些？你的项目怎么用的？

参考回答：

组件	作用	项目应用
Nacos	注册中心 + 配置中心	服务注册发现、配置热更新
Spring Cloud Gateway	API 网关	统一鉴权、路由转发、限流
OpenFeign	声明式调用	服务间 HTTP 调用
Sentinel	熔断降级	第三方服务熔断、登录接口限流
Seata	分布式事务	了解，未深度使用

Q3: Nacos 作为注册中心和配置中心，和 Eureka/Apollo 有什么区别？

参考回答：

- **Nacos vs Eureka**：Nacos 支持 AP/CP 切换，Eureka 仅 AP；Nacos 支持配置管理，Eureka 不支持；Nacos 支持权重路由，Eureka 没有
 - **Nacos vs Apollo**：Nacos 轻量级、部署简单；Apollo 功能更丰富但更重。中小项目 Nacos 注册+配置一体化更方便
-

Q4: Spring Cloud Gateway 怎么做统一鉴权？

参考回答：

通过 GlobalFilter 实现：

```
@Component
public class AuthFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String token = exchange.getRequest().getHeaders().getFirst("Authorization");
        if (StringUtils.isBlank(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
        // 校验 JWT
        Claims claims = JwtUtil.parseToken(token);
        // 透传用户信息到下游
        ServerHttpRequest request = exchange.getRequest().mutate()
            .header("X-User-Id", claims.getSubject())
            .header("X-Tenant-Id", claims.get("tenantId", String.class))
            .build();
        return chain.filter(exchange.mutate().request(request).build());
    }

    @Override
    public int getOrder() { return -1; }
}
```

项目关联：API 网关项目中，用 Gateway + GlobalFilter 实现 JWT 校验和租户信息透传。

Q5：OpenFeign 调用超时怎么处理？

参考回答：

1. 配置超时时间：feign.client.config.default.connectTimeout=5000
2. 开启 Sentinel 整合：@FeignClient(name="xxx", fallbackFactory=XxxFallbackFactory.class)
3. Fallback 降级处理：返回默认值或抛出业务异常
4. 重试策略：Ribbon/LoadBalancer 配置重试次数

```
feign:
  client:
    config:
      default:
        connectTimeout: 5000
        readTimeout: 10000
  sentinel:
    enabled: true
```

二、高并发与限流

Q6：Redis 滑动窗口限流怎么实现？

参考回答：

通过 Redis ZSET + Lua 脚本保证原子性：

```
// Lua 脚本
String LUA_SCRIPT = """
    local key = KEYS[1]
    local now = tonumber(ARGV[1])
    local window = tonumber(ARGV[2])
    local limit = tonumber(ARGV[3])
    redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
    redis.call('ZADD', key, now, now)
    redis.call('EXPIRE', key, window / 1000 + 1)
    local count = redis.call('ZCARD', key)
    if count > limit then
        return 0
    end
    return 1
""";

Long result = redisTemplate.execute(
    new DefaultRedisScript<>(LUA_SCRIPT, Long.class),
    List.of("rate:limit:" + phone),
    System.currentTimeMillis(), 60000, 5
);
```

项目关联：考试系统登录限流，按手机号+IP+设备指纹多维度分桶。

Q7: Sentinel 怎么做熔断降级？和 Hystrix 区别？

参考回答：

Sentinel 三种熔断策略：

1. **慢调用比例：**响应时间超过阈值的比例达到阈值时熔断
2. **异常比例：**异常比例达阈值时熔断
3. **异常数：**异常数达阈值时熔断

Sentinel vs Hystrix：

- Sentinel 支持更细的控制（QPS、线程数、关联、链路等），Hystrix 主要是线程池隔离+信号量
 - Sentinel 有控制台实时监控，Hystrix 需要 Turbine 聚合
 - Sentinel 持久化规则到 Nacos，Hystrix 配置相对固定
-

三、缓存与一致性

Q8: 延迟双删策略是什么？为什么要延迟？

参考回答：

更新数据库时的缓存策略：

1. 先删除缓存
2. 再更新数据库
3. 延迟一段时间（如 200ms）后再次删除缓存

为什么要延迟：防止并发场景下，步骤 2 执行期间有读请求将旧值重新写入缓存。延迟时间需大于一次读请求的耗时。

```

public void updateInvoice(Invoice invoice) {
    // 1. 先删缓存
    redisTemplate.delete("invoice:" + invoice.getId());
    // 2. 更新数据库
    invoiceMapper.updateById(invoice);
    // 3. 延迟双删
    CompletableFuture.delayedExecutor(200, TimeUnit.MILLISECONDS)
        .execute(() -> redisTemplate.delete("invoice:" + invoice.getId()));
}

```

项目关联：电子发票网关中保证缓存与数据库一致性。

Q9：缓存穿透、缓存击穿、缓存雪崩怎么解决？

问题	原因	解决方案
缓存穿透	查不存在的数据，请求直达数据库	布隆过滤器 / 缓存空值
缓存击穿	热点 key 过期，大量请求打到数据库	互斥锁（Redisson） / 逻辑过期
缓存雪崩	大量 key 同时过期	过期时间加随机值 / 多级缓存 / 永不过期+异步刷新

Q10：Redis 分布式锁用 Redisson 怎么实现？

参考回答：

```

RLock lock = redissonClient.getLock("invoice:lock:" + channelCode);
try {
    // 尝试加锁，等待 5 秒，锁自动释放 30 秒
    if (lock.tryLock(5, 30, TimeUnit.SECONDS)) {
        // 业务逻辑：开票
        invoiceService.issue(request);
    }
} finally {
    if (lock.isHeldByCurrentThread()) {
        lock.unlock();
    }
}
}

```

Redisson 优势：

- 看门狗自动续期，防止业务未完成锁就释放
- 可重入锁
- 自带 RedLock 算法（集群模式）

项目关联：多渠道开票用分布式锁串行化，避免并发导致状态异常。

四、消息队列

Q11：RabbitMQ 怎么保证消息不丢失？

参考回答：

三个环节保证：

1. 生产端确认：publisher-confirm-type=correlated，消息到达交换机回调确认
2. 交换机到队列：publisher-returns=true，消息无法路由到队列时回调
3. 消费端手动 ACK：

```
@RabbitListener(queues = "transcode.queue")
public void handleTranscode(TranscodeJob job, Channel channel,
                             @Header(AmqpHeaders.DELIVERY_TAG) long tag) throws
IOException {
    try {
        transcodeService.process(job);
        channel.basicAck(tag, false);
    } catch (Exception e) {
        channel.basicNack(tag, false, true); // 重回队列
    }
}
```

项目关联：透析机数据采集用 RabbitMQ，消息持久化 + 手动 ACK 确保不丢失。

Q12: RabbitMQ 和 RocketMQ 怎么选？

维度	RabbitMQ	RocketMQ
语言	Erlang	Java
吞吐量	万级	十万级
延迟	微秒级	毫秒级
事务消息	不原生支持	支持
适用场景	中小规模、业务解耦	大规模、金融级、顺序消息
生态	Spring AMQP 原生支持	阿里生态

项目选型：考试系统用 RabbitMQ（规模适中、Spring 生态友好）；如果涉及金融级事务消息可考虑 RocketMQ。

五、数据库与 MyBatis-Plus

Q13: MyBatis-Plus 多租户怎么实现？

参考回答：

通过 TenantLineInnerInterceptor 自动拦截 SQL：

```
@Bean
public MybatisPlusInterceptor mybatisPlusInterceptor(TenantManager tenantManager) {
    MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
    interceptor.addInnerInterceptor(new TenantLineInnerInterceptor(
        () -> new LongValue(tenantManager.getCurrentTenantId())
    ));
    return interceptor;
}
```

- 所有 SELECT/INSERT/UPDATE/DELETE 自动追加 tenant_id = ? 条件
- 特定表可配置忽略租户过滤（如 @InterceptorIgnore(tenantLine = "true")）

项目关联：HDIS 系统通过租户插件实现数据隔离，与 .NET 端 SqlSugar 的 TenantFilter 思路一致。

Q14：深度分页怎么优化？

参考回答：

问题：LIMIT 100000, 10 会扫描 100010 行再丢弃前 100000 行

方案一：游标分页（推荐）

```
-- 基于 ID 游标
SELECT * FROM exam_record WHERE id > 100000 ORDER BY id LIMIT 10;
```

方案二：子查询优化

```
SELECT * FROM exam_record
WHERE id >= (SELECT id FROM exam_record ORDER BY id LIMIT 100000, 1)
LIMIT 10;
```

项目关联：考试系统大量考试记录查询，改用游标分页避免深度分页问题。

Q15：MyBatis-Plus 和 JPA 怎么选？

维度	MyBatis-Plus	JPA
SQL 控制	完全控制	框架生成
学习曲线	低	中
复杂查询	灵活	需手写 JPQL/Criteria
适合场景	复杂业务、报表多	简单 CRUD、领域驱动

项目选型：医疗系统复杂查询多（报表、统计），用 MyBatis-Plus 更灵活。

六、设计模式

Q16：工厂模式 + 策略模式在项目中怎么用？

参考回答：

工厂模式：创建对象，解耦创建与使用

```
// 统一接口
public interface InvoiceClient {
    InvoiceResult issue(InvoiceRequest request);
}

// 工厂 + Spring 注入
@Service
public class InvoiceClientFactory {
    private final Map<String, InvoiceClient> clients;

    public InvoiceClientFactory(Map<String, InvoiceClient> clients) {
        this.clients = clients;
    }
}
```

```

    }

    public InvoiceClient getClient(String vendor) {
        return Optional.ofNullable(clients.get(vendor))
            .orElseThrow(() -> new BusinessException("不支持的渠道: " + vendor));
    }
}

```

策略模式：运行时切换算法

```

// 金融保函风控策略
public interface RiskStrategy {
    RiskResult evaluate(GuaranteeRequest request);
}

@Component("bankA")
public class BankARiskStrategy implements RiskStrategy { /* ... */ }

@Component("bankB")
public class BankBRiskStrategy implements RiskStrategy { /* ... */ }

```

七、雪花算法与分布式 ID

Q17：雪花算法 ID 重复怎么办？

参考回答：

雪花算法结构：1 位符号 + 41 位时间戳 + 10 位机器 ID + 12 位序列号

冲突场景：时钟回拨、机器 ID 重复

解决方案：

1. 机器 ID 由 Nacos 注册中心分配，避免重复
2. 时钟回拨检测：回拨小于 5ms 等待，大于 5ms 抛异常
3. MyBatis-Plus 的 `IdWorker` 已内置回拨处理

```

mybatis-plus:
  global-config:
    db-config:
      id-type: assign_id # 雪花算法

```

八、接口幂等性

Q18：如何保证接口幂等性？

参考回答：

方案	实现	适用场景
Redis SETNX	请求带唯一流水号，SETNX 判断	交卷、开票
数据库唯一索引	业务唯一键建唯一索引	订单创建
Token 机制	先获取 token，提交时校验并删除	表单提交

方案	实现	适用场景
状态机	只允许特定状态流转	订单状态变更

项目应用：

- 考试交卷：Redis SETNX + 考生 ID + 试卷 ID
- 开票请求：Redis SETNX + 业务流水号

九、文件上传与转码

Q19：大文件分片上传怎么做？

参考回答：

1. 前端切片：File.slice() 分片 + 计算文件 MD5
2. 秒传判断：服务端根据 MD5 判断是否已存在
3. 分片上传：并行上传各分片，支持断点续传
4. 合并确认：全部分片上传完成后服务端合并

```
@PostMapping("/upload/chunk")
public Result uploadChunk(@RequestParam String fileId,
                          @RequestParam int chunkIndex,
                          @RequestParam MultipartFile file) {
    // 保存分片到临时目录
    chunkService.saveChunk(fileId, chunkIndex, file);
    return Result.success();
}

@PostMapping("/upload/merge")
public Result mergeChunks(@RequestParam String fileId,
                          @RequestParam String fileName) {
    chunkService.mergeChunks(fileId, fileName);
    return Result.success();
}
```

Q20：视频转码怎么做异步处理？

参考回答：

RabbitMQ + FFmpeg 异步转码：

```
// 生产者：提交转码任务
rabbitTemplate.convertAndSend("transcode.exchange", "transcode.route",
    new TranscodeJob(fileId, sourcePath, "mp4"));

// 消费者：异步处理
@RabbitListener(queues = "transcode.queue")
public void handleTranscode(TranscodeJob job) {
    String cmd = String.format("ffmpeg -i %s -c:v libx264 %s",
        job.getSourcePath(), job.getOutputPath());
    Runtime.getRuntime().exec(cmd);
    // 转码完成回调
}
```



```
        callbackService.notify(job.getFileId(), Status.COMPLETED);  
    }
```

效果：转码等待时长下降约 40%。

十、面试反问准备

Q21：你有什么问题想问面试官的？

建议问题：

1. 贵司微服务架构用的哪套组件？Spring Cloud Alibaba 还是原版？
 2. 这个岗位主要负责哪个业务模块？需要对接哪些第三方服务？
 3. 团队目前的并发规模大概是多少？有没有遇到什么技术挑战？
 4. 有没有代码规范、Code Review、自动化测试等工程实践？
 5. 项目是用 Spring Boot 单体还是 Spring Cloud 微服务？未来的演进方向是什么？
-

建议：结合简历中的项目，选择 2-3 个最深入的场景重点准备，确保能回答出"怎么做的"和"为什么这样做"。Java 面试特别关注 Spring Cloud 全家桶的理解深度，务必熟悉 Nacos、Gateway、Sentinel、OpenFeign 的核心原理和配置方式。